Accelerating Image Preprocessing with CUDA: High-Speed Gaussian Filtering and Brightness Enhancement

Mekhriddin Rakhimov
Department of Computer Systems
Tashkent University of Information Technologies named after Muhammad Al-Khwarizmi
Tashkent, Uzbekistan
raximov022@gmail.com

Shakhzod Javliev
Department of Computer Systems
Tashkent University of Information Technologies named after Muhammad Al-Khwarizmi
Tashkent, Uzbekistan
shajavliyev@gmail.com

2025 IEEE 26th International Conference of Young Professionals in Electron Devices and Materials (EDM) | 978-1-6654-7737-6/25/$31.00 ©2025 IEEE | DOI: 10.1109/EDM65517.2025.11096877

***Abstract*—These days, we can see that as technology advances, so does the volume of data. Furthermore, it takes a long time to perform digital operations on image-based data. This article examines issues related to computing devices' speed at processing massive volumes of data. Using one of the image preprocessing techniques, the Gaussian filter, the study's primary objective is to speed up the processes of enhancing image brightness and eliminating noise. This is accomplished by processing images of various sizes in parallel on the computer's graphics processor using CUDA (Compute Unified Device Architecture) technology, as well as on the central processor using the OpenMP (Open Multi-Processing) parallel library for devices without a graphics processor. The study concludes with the presentation of the percentage increase indicators of CUDA technology in comparison to OpenMP when preprocessing images of sizes 512x512, 1024x1024, and 3200x2400 using these parallel processing technologies. At the end of the study, the capabilities of CUDA technology in image processing are highly evaluated compared to other parallel processing tools. OpenMP technology is proposed for parallel image processing on devices without a graphics processor.**

Keywords—heterogeneous computing systems, CPU, GPU, CUDA, OpenMP, parallel processing, image processing.

I. INTRODUCTION

It is known that the rapid growth of data and its volume as a result of the development of artificial intelligence is causing problems with the speed of data processing and calculation processes of modern computers [1]. Although multi-core processors are being developed today, there are still problems that require long or intensive computing efforts when processing and analyzing large amounts of data. Although Gordon Moore, one of the founders of Intel, predicted in a 1965 paper that the number of transistors would double every year for the next 10 years [2]. However, due to some limitations with the development of technology today, as well as the sharp increase in the volume of data, data processing on computing devices takes a long time. Especially when working with large image or video datasets, it is more efficient to use a graphics processing unit (GPU) than to rely on the computer's central processing unit (CPU) to perform calculations, because GPUs have thousands of computational units compared to CPUs, making these computational units more suitable for large tasks [3]. In traditional Von-Neumann computing devices, data processing and calculations were performed by the computer's central processor, whose main task was arithmetic/logical processing, process control, and input/output operations [4]. When working with images or videos on a computer, it is preferable to use the graphics processor instead of the main processor to process the images or videos. Using a GPU rather than the CPU is frequently more efficient when processing images or working with large amounts of data. This is because of the architecture of the GPU, which was created especially for these kinds of tasks.

The GPU's multicore and parallel processing capabilities are more appropriate for large-scale data processing and video analysis, while the CPU manages arithmetic, logical, control, and input/output operations in accordance with program instructions. Figure 1 shows the main distinctions between CPU and GPU processing cores (Fig. 1).

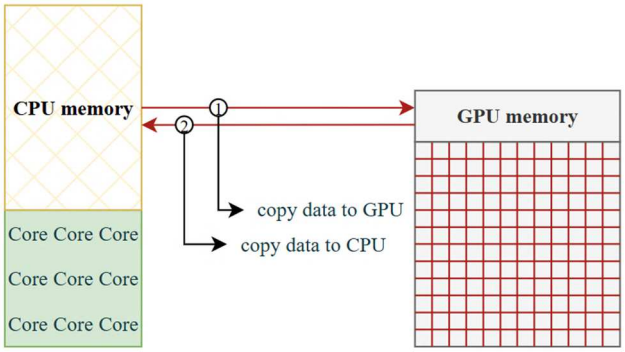


Fig. 1. Interconnection and data exchange between CPU and GPU.

From the image above, we can see that the GPU has thousands of cores compared to the CPU, but at the same time, the GPU downloads data from the CPU's memory to its own memory to process data and processes the data based on the instructions given by the CPU. Also, the more data is loaded into the GPU's memory, the more speed problems are observed in the GPU, and to overcome this problem, parallel processing on the GPU is desirable.

II. RELATED WORKS

Large data set analysis is known to take a lot of time and computational power, but high-precision parallel processing techniques and technologies are currently successfully completing these tasks [5], [6]. As the amount of big data increases, mathematical calculations on this data also become more difficult [7], [8]. We can also observe a sharp increase in the volume of data in the medical field [9], [10], [11].

Taking images as an example, using GPUs to work with such data is beneficial for achieving computational speed rather than relying solely on the main central processing unit of the computer. Since the main task of many modern computer GPUs is to calculate large amounts of data and process graphic images or videos in a program, speed problems arise when processing video and images [12], [13]. Although GPUs are designed to solve such problems, as the data size increases, processing even on GPUs takes a long time. In some cases, speed can be achieved through parallelization. Since we are talking about images, their processing technologies must also be more efficient. This parallelization process, especially when implemented on GPU architecture, greatly simplifies and speeds up image processing tasks [14], [15]. Based on our knowledge from the studies we have studied, the use of GPUs in image processing significantly increases the speed [16]. In this case, speed can be achieved by using special heterogeneous computing systems that distribute computational tasks between several processors, leading to fast and efficient results.

In this study, we will consider special parallel processing technologies related to heterogeneous computing systems for image processing.

III. Parallel Processing on the CUDA Architecture

Currently, high-performance computing in data processing in various fields requires the use of parallel computing tools. Parallel computing is becoming a major trend for high-performance computing on multi-core processors, and in the development of processors, rather than increasing their speed, it is required to increase the number of cores, which in turn ensures parallelism in computing devices [17]. In order to ensure high parallelism when processing massive volumes of data, GPUs are crucial. NVIDIA created a unique parallel computing technology called CUDA (Compute Unified Device Architecture) to process data in parallel on NVIDIA GPUs [18]. For non-NVIDIA GPUs, OpenCL (Open Computing Language) technology was created by the Khronos Group [19]. The Single Instruction Multiple Data (SIMD) architecture is the foundation of the GPU, a processor that performs well at accelerating 3D graphics [20]. It is primarily used to speed up graphics processing, image processing, and video processing. It is made to execute SIMD tasks, which are one instruction, multiple data streams. This is because the processor that supports the execution of image processing algorithms is the graphics processor GPU. But GPUs often face high demands when processing large volumes of images or videos. We can improve time efficiency in these operations by encouraging parallel processing and utilizing heterogeneous computing systems [14], [16]. Multiple processors or cores are used in heterogeneous computing systems, and different kinds of coprocessors are integrated in addition to identical processors to achieve performance and energy efficiency.

In the heterogeneous computing system in the CUDA architecture, which is a parallel engine for the GPU, the device code runs on the GPU, while the host code runs on the central processor, so the processor is called the host and the GPU is called the device. Accordingly, a heterogeneous system is one that combines various multi-core processor types to execute parallel processing. In this instance, the computer's CPU and GPUs cooperate to boost speed (Fig. 2).

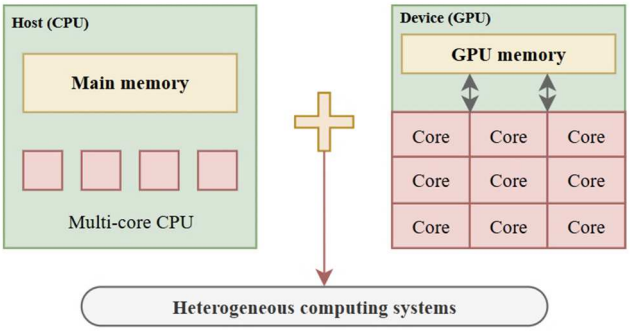


Fig. 2. Heterogeneous computing systems.

It is currently believed that NVIDIA's GPUs are the most efficient. To parallelize this GPU device, CUDA technology is recommended. Therefore, as mentioned above, one of the instrumental tools of heterogeneous computing systems supported by graphics processing units is the CUDA technology for parallel image processing. In this case, data from the CPU is loaded into the GPU's memory and distributed among the GPU's core blocks using the CUDA model. Therefore, CUDA is an API model created by Nvidia for parallel computing and an extension of the C programming language [21]. To improve overall computing performance, programs created with CUDA make use of a computer's GPU computational power. In this sense, image processing using CUDA technology can produce positive outcomes.

IV. Parallel Image Processing with CUDA Technologies

The term "image processing" refers to a variety of tools and methods used to examine, edit, and enhance digital photos in order to extract useful information or transform them into a desired format. The manipulation or analysis of visual data recorded as images is referred to as image processing. It entails using algorithms to improve, alter, or extract data from pictures. Filtering, edge detection, segmentation, and noise reduction are typical image processing tasks [22], [23]. In domains like satellite imaging, computer vision, medical imaging, and photography, image processing techniques are extensively employed. The primary prerequisite for discussing images and their processing is familiarity with images and their processing techniques. Analyzing, altering, improving, extracting information from, or transforming digital images into a desired form are the main definitions of image processing. Digital image processing is a field in which algorithms and techniques are developed to improve certain properties of digital images, the most popular of which are filtering, rasterization, segmentation, and noise removal [24]. A processed image or other useful outcome can be the output of image processing, which is a type of signal processing where the input data can be images or video data. For example, converting a color image to a grayscale image, smoothing an image and removing unnecessary noise, extracting necessary features from images, and enhancing the brightness of grayscale images are the main types of image processing [25]. Even improving an outdated (damaged) image is an example of image processing [26], [27].

In this study, we will consider the Gaussian Blur filter as an example of image processing and implement methods for enhancing image brightness, one of the broad processes used in image processing.

A. Gaussian Filtering

Gaussian Blur is a widely used filtering algorithm in image processing that reduces noise and smoothes images [28]. This algorithm permits pixels to have different weights according to their distance from the center by using a filter based on the Gaussian function. As a result, pixels that are closer to the center are more important, while pixels that are farther from the center are less so. Both color and grayscale images may have a lot of noise, or sporadic changes in pixel color or brightness. The high standard deviations of the pixels in these pictures simply indicate that there is a great deal of variation among pixel groups. Gaussian blur, also called convolution, creates a third function by combining two mathematical functions (one for the x-axis and one for the y-axis) because a photograph is two-dimensional. This process can usually be done by applying a low-pass filter to the image, where the low-pass filter reduces high-frequency values while preserving low-frequency values, resulting in a smoothed image by reducing the differences between nearby pixels. This is the most commonly used image smoothing process to reduce noise and detail in an image. The following formula describes the Gaussian Blur function to reduce the noise in an image.

$$G_{(x,y)} = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}} \quad (1)$$

where x and y are the distances of the pixels from the center, and σ is the degree of influence of the filter. In this case, each pixel is replaced by the average of its own and the weights of the surrounding pixels, and the weights are taken from the Gaussian function, with the nearest pixels given a larger weight and the farthest pixels given a smaller weight. In the figure below, we can see an image that has been de-noised using a Gaussian filter (Fig. 3).

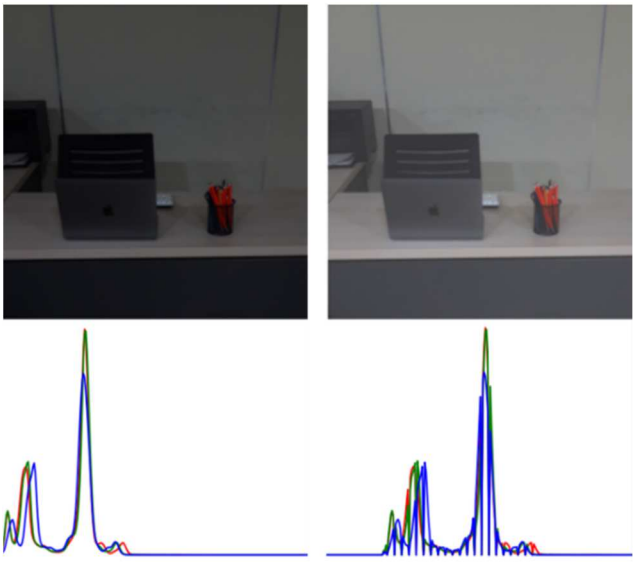
(a) Before filtering *(b) After filtering*

Fig. 3. Gaussian filtering.

Gaussian blur does not always preserve detail, as it can also blur the edges of an image, as well as dusting off noise. Therefore, it is recommended to use it wisely.

B. Image Histogram Equalization

In image analysis and processing, a histogram is a graph that displays the distribution of pixel values in an image [29]. In terms of brightness or color intensity, it displays the number of pixels in the image. Each pixel in a grayscale image has a brightness value between 0 and 255. In color images, each color channel is calculated separately for red, green, and blue (RGB). For example, if most of an image is dark, its histogram will be skewed to the left (close to the 0 value), or vice versa, that is, in a bright image, the histogram will be skewed to the right (close to the 255 value). In the figure below, we can see a dark image processed using the Gaussian filter method and an image with its brightness increased (Fig. 4).



(a) Before equalizing *(b) After equalizing*

Fig. 4. Image histogram equalization.

Image enhancement is a common process in image processing, where the brightness value of pixels is increased. It is usually used to make an image sharper and more visible. Making images brighter or darker is an example of a common image enhancement tool available in most image editors. Often, the processing occurs on the entire image, with the same steps applied to each pixel of the image. This means that the same work is repeated many times. New technologies allow for better quality images.

C. Image Processing Using CUDA Technology

Using CUDA technology, we use the computer's GPU device for parallel processing of images, but in the GPU, a certain operation for image processing is offloaded to the CPU, therefore, the image processing flow using CUDA technology is carried out in the following stages, in which some tasks are offloaded from the processor and performed by the GPU: first, the GPU loads the data from the main memory to its memory. In the second stage, the processor provides the GPU with processing instructions. In the third stage, it processes the images loaded into the GPU memory. Finally, the result is returned from the GPU memory to the main memory in the fourth stage. To see these stages more clearly, let's pay attention to the following figure (Fig. 5).

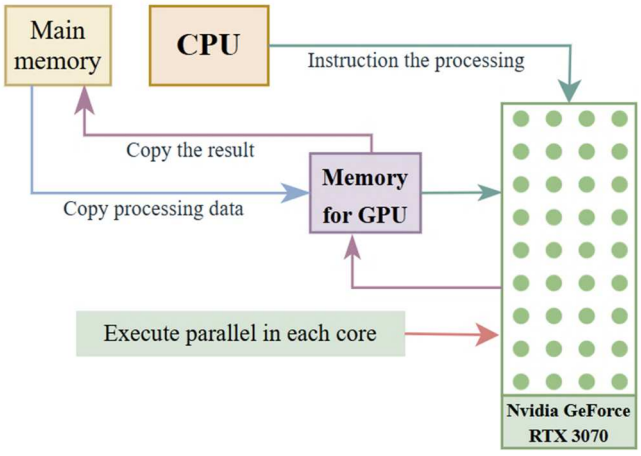
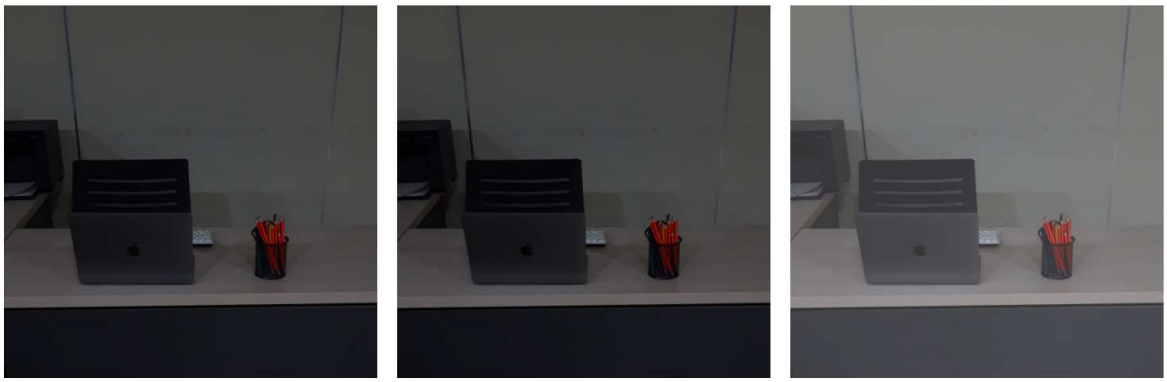


Fig. 5. Structure of the image processing stages and GPU.

Image processing jobs do not require complete CUDA programming, as seen in Fig. 5. Parallel programming helps important specialists by dividing the work that the software needs to do into smaller, easier-to-manage pieces that can be finished simultaneously. Distributing the data to the computer nodes achieves this. While CUDA enables the GPU to process the data in parallel, distributed on the CPU may be handled serially. Thus, when the data is distributed among the compute nodes, this process is executed serially by CPU nodes and in parallel by GPU nodes. Although it may take developers longer to write efficient parallel algorithms and code, parallel programming reduces numerical processing time because it operates concurrently on multiple compute nodes and processor cores.

In the image below, we can see the results of the above Aimage processing methods (Fig. 6).



(a) Original image *(b) After Gaussian filtering* *(c) After equalizing*

Fig. 6. Image processing with CUDA technology.

V. Accelerating Gaussian Filtering with Parallel Processing Technologies

The characteristics of our result in Figure 3 and Figure 4 may not differ much from the CUDA-processed image in Figure 6. The CUDA parallel processing technology on the GPU makes it possible for modern computers to run high-resolution graphics. However, since not all devices have a GPU, it is advisable to implement parallel processing using other technologies. For example, CUDA technology can be used for parallel processing for NVIDIA GPUs, while OpenCL technology for heterogeneous computing systems can be effective for other types of GPUs (non-NVIDIA GPUs). If our computing devices do not have a GPU at all, then all processing operations are offloaded to the CPU, which can slow down the process.

In this research work, we focused on accelerating the application of Gaussian filtering in image preprocessing, but we also considered devices without GPUs and studied the capabilities of OpenMP technology for CPUs.

A. Accelerating Gaussian Filtering with CUDA

The grid and block sizes chosen with CUDA technology are known to affect GPU parallel computing efficiency. The Grid, which is a shared set of blocks with multiple threads, is the cornerstone of CUDA's parallel computing. in order to calculate each pixel or component in the image using a thread and separate executable code. The main goal of this research is to pre-process images using Gaussian filtering. This filtering process uses CUDA technology to perform different CUDA blocks on the GPU, and each thread in the block affects the reversibility of the image filtering process. Therefore, we used different block sizes to perform these processes and achieved faster results in these processes. We experimented with the following block sizes for this study: 8x8 blocks are small, 16x16 blocks are medium, and 32x32 blocks are large. Additionally, we obtained the following speed outcomes (Table I). The findings demonstrate that employing large blocks (32x32) in CUDA-based image processing greatly boosts computational efficiency.

TABLE I. Results of the Gaussian Blur using CUDA

The incoming image's size	Block size 8x8 (ms)	Block size 16x16 (ms)	Block size 32x32 (ms)
512x512	3.1	2.3	1.28
1024x1024	9.2	7.3	3.95
3200x2400	64.7	34.1	21.8

B. Accelerating Gaussian Filtering with OpenMP

Since CUDA only supports NVIDIA GPUs, not all of the computers used for the study had a GPU. As a result, OpenMP (Open Multi-Processing) and other parallel processing technologies work well in non-GPU computing processes. The study also looked at the potential of CUDA technology for image processing. The reason for using OpenMP is that it's a useful tool for devices without a GPU [30]. Despite not using grids and blocks like CUDA, OpenMP technology has its own processing threads. Since we are working on the CPU and OpenMP technology only supports the CPU, we can manage the threads appropriately to get a result that is comparable to the blocks in CUDA. If we use 8, 16, or 32 threads using OpenMP technology, this has a similar effect to increasing the block size in CUDA, that is, in OpenMP, the number of threads is increased instead of the Block Size, and accordingly, the execution of computational processes becomes much faster (Table II).

TABLE II. Results of the Gaussian Blur using OpenMP

The incoming image's size	8 thread (ms)	16 thread (ms)	32 thread (ms)
512x512	44.1	34.78	19.1
1024x1024	144.93	114.3	58.9
3200x2400	971.2	504.3	324.1

Using OpenMP to generate more parallel streams significantly boosts speed, however this is restricted by the CPU's finite number of cores.

VI. Experimental Results

The study found that CUDA technology can result in more efficient image processing because GPUs have more computational units than CPUs. But for devices without GPUs, we can use OpenMP technology to fix the speed problems. The CPU can process data in parallel thanks to OpenMP's user-defined stream control, even though it lacks the same units as CUDA. Although this study focuses on parallel image processing on GPUs, we have also investigated methods to implement parallel capabilities on non-GPU devices. The features of our research device are compiled in the following table (Table III).

TABLE III. CHARACTERISTICS OF PROCESSORS

№	Model processor	Cores / Threads	Cache memory L1 / L2 / L3	Memory Bandwidth	Parallel technologies
1	Intel Core i9-13900H	24/32	2 MB / 32 MB / 36 MB	-	OpenMP (5.0 version)
	Nvidia GeForce RTX 3070	5120	GPU memory (8GB GDDR6 VRAM)	384.0 GB/s	CUDA (12.8 version)

As everyone knows, GPUs can perform various data processing jobs far more quickly than a computer's CPU. Parallel programming is fundamentally much faster than sequential processing because it divides complex issues into smaller, easier-to-manage jobs that may be finished concurrently with numerous computing resources. In this research, we focused on image processing speed. We can observe that CUDA technology, which uses parallel processing on the GPU, is far more efficient than other technologies when it comes to computing speed. Although the goal of the study was to investigate the potential of CUDA technology, not all devices have NVIDIA GPUs or even a graphics processor.

As a result of the research, we proposed an OpenMP framework for parallel processing on devices without a GPU. The table IV below shows the comparative values of the inverse values achieved in image preprocessing using the advances achieved during the research (i.e. CUDA for GPUs, OpenMP for CPUs on machines without a GPU).

TABLE IV. PERFORMANCE COMPARISON OF GAUSSIAN FILTERING: CUDA VS OPENMP ACCELERATION

The incoming image's size	CUDA (Block size)			OpenMP (Threads)			Percent Decrease (%)
	8x8 (ms)	16x16 (ms)	32x32 (ms)	8 (ms)	16 (ms)	32 (ms)	
512x512	3.1	2.3	1.28	44.1	34.78	19.1	~ **93.03**
1024x1024	9.2	7.3	3.95	144.93	114.3	58.9	~ **94.09**
3200x2400	64.7	34.1	21.8	971.2	504.3	324.1	~ **93.27**

To determine the difference in speed between CUDA and OpenMP, Percent Decrease is calculated using the following formula:

$$Percent\ Decrease = \frac{OpenMP_{time} - CUDA_{time}}{OpenMP_{time}} \times 100 \qquad (2)$$

Here: $OpenMP_{time}$ is the time spent on the processor using OpenMP (usually measured in milliseconds). $CUDA_{time}$ is the time taken to perform the calculation using CUDA technology. $Percent\ Decrease$ is much less time $CUDA_{time}$ consumes than $OpenMP_{time}$.

A graphical depiction of the speeds given in the Table IV above can be seen in the figure below (Fig. 7).

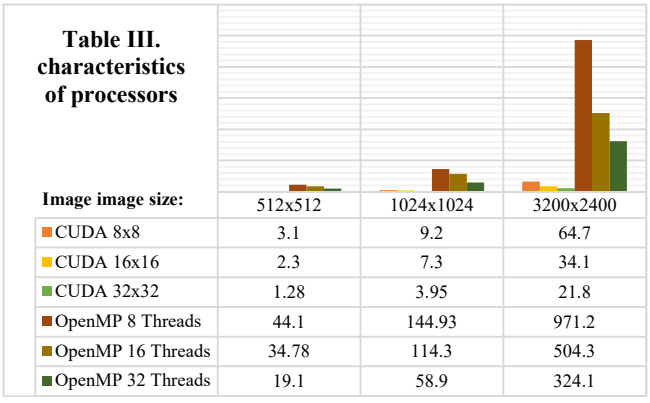


Fig. 7. CUDA vs OpenMP acceleration: an illustration of Table IV.

The results of the study in Table IV above show that CUDA technology consumes up to 94% less time than OpenMP parallel processing engine in image processing. This percentage is derived using Formula 2, which compares CUDA with 8x8x, 16x16, 32x32 block pointers and OpenMP with 8, 16, 32 threads. And their approximate arithmetic mean is obtained.

VII. Conclusion

In this work, we examined CUDA technology, one of the essential elements of heterogeneous computing systems. Finally, CUDA technology can yield results significantly faster than anticipated when used in the initial stages of image processing. According to the study's findings, CUDA technology outperforms OpenMP parallel processing tools by approximately 15 times during image processing procedures, saving up to 94% of the time.

As a result, CUDA technology's image processing capabilities are highly assessed in comparison to other parallel processing tools. But if we consider that CUDA technology is limited to NVIDIA GPUs, OpenMP technology is a useful tool for processing images in parallel on devices without a GPU. Our evaluation of CUDA technology for devices with NVIDIA GPUs in parallel image processing is based on the research findings that have been obtained and examined. We suggest OpenCL technology for other GPU types and OpenMP parallel processing tools for devices without any GPUs, since not all devices have NVIDIA GPUs.

VIII. Future Research Work

In this study, we investigated the performance of CUDA technology in image preprocessing compared to OpenMP parallel processing technologies. In future studies, we will focus on medical image processing and the capabilities of OpenCL technology for other types of graphics processors.

References

[1] M. Rakhimov, S. Javliev, and R. Nasimov, "Parallel Approaches in Deep Learning: Use Parallel Computing," 7th International Conference on Future Networks and Distributed Systems, Dubai United Arab Emirates: ACM, Dec. 2023, pp. 192–201. doi: 10.1145/3644713.3644738.

[2] Intel, "Moore's Law," Intel Newsroom, [Online]. Available: https://newsroom.intel.com/press-kit/moores-law.

[3] C. Yu and M. Cai, "An Image Depth Processing Method Based On Parallel Computing and Multi-GPU," 2021 2nd International Conference on Smart Electronics and Communication (ICOSEC), Trichy, India, 2021, pp. 1009-1012, doi: 10.1109/ICOSEC51865.2021.9591686.

[4] SoftwareG, "How to use GPU to help CPU," [Online]. Available: https://softwareg.com.au/blogs/computer-hardware/how-to-use-gpu-to-help-cpu.

[5] Nan Zhang, Yun-shan Chen and Jian-li Wang, "Image parallel processing based on GPU," 2010 2nd International Conference on Advanced Computer Control, Shenyang, China, 2010, pp. 367-370, doi: 10.1109/ICACC.2010.5486836.

[6] I. Khujayorov and M. Ochilov, "Parallel Signal Processing Based-On Graphics Processing Units," 2019 International Conference on Information Science and Communications Technologies (ICISCT), Tashkent, Uzbekistan, 2019, pp. 1-4, doi: 10.1109/ICISCT47635.2019.9011976.

[7] M. Musaev, D. Juraev, M. Ochilov, and M. Abdullaeva, "Text processing technology in Uzbek speech to sign language translation systems", Samarkand, Uzbekistan, 2024, p. 030062. doi: 10.1063/5.0241738.

[8] U. Turdiyev and K. Imomnazarov, "A system of equations of the two-velocity hydrodynamics without pressure," AIP Conference Proceedings, vol. 2365, p. 070002, Jan. 2021, doi: 10.1063/5.0058372.

[9] K. Zohirov, G. Berdiev, S. Boykobilov, G. Pardayeva and M. Ochilov, "Kinematic and Kinetic Analysis to Improve the Overall Fitness of Wrestlers," 2024 4th International Conference on Technological Advancements in Computational Sciences (ICTACS), Tashkent, Uzbekistan, 2024, pp. 676-681, doi: 10.1109/ICTACS62700.2024.10840561.

[10] K. Zohirov, G. Berdiev, S. Boykobilov, K. Egamberdiev and G. Pardayeva, "Using the TUG Test in Implementing Gait Analysis of Different Types of Wrestlers," 2024 4th International Conference on Technological Advancements in Computational Sciences (ICTACS), Tashkent, Uzbekistan, 2024, pp. 1113-1120, doi: 10.1109/ICTACS62700.2024.10841194.

[11] A. Abdusalomov, M. Mukhiddinov, O. Djuraev, U. Khamdamov, and U. Abdullaev, "AI-Based Estimation from Images of Food Portion Size and Calories for Healthcare Systems," in Lecture notes in computer science, 2024, pp. 9–19. doi: 10.1007/978-3-031-53830-8_2.

[12] M. Umarov, J. Elov, S. Khalilov, I. Narzullayev and M. Karimov, "An algorithm for parallel processing of traffic signs video on a graphics processor," 2022 International Conference on Information Science and Communications Technologies (ICISCT), Tashkent, Uzbekistan, 2022, pp. 1-5, doi: 10.1109/ICISCT55600.2022.10146809.

[13] T. A. Kuchkorov, J. X. Djumanov, T. D. Ochilov, and N. Q. Sabitova, "Satellite imagery super resolution using classical and deep learning algorithms," in Lecture notes in computer science, 2024, pp. 70–80. doi: 10.1007/978-3-031-53830-8_8.

[14] M. Rakhimov, D. Zaripova, S. Javliev, and J. Karimberdiyev, "Deep learning parallel approach using CUDA technology", Samarkand, Uzbekistan, 2024, p. 030003. doi: 10.1063/5.0241439.

[15] C. Yu, S. Royuela, and E. Quiñones, "Enhancing heterogeneous computing through OpenMP and GPU graph," in Proc. 53rd Int. Conf. Parallel Process. (ICPP '24), New York, NY, USA: ACM, 2024, pp. 534–543. doi: 10.1145/3673038.3673050.

[16] R. Nasimov, M. Rakhimov, S. Javliev, and M. Abdullaeva, "Parallel approaches to accelerate deep learning processes using heterogeneous computing," in Lecture notes in computer science, 2024, pp. 32–41. doi: 10.1007/978-3-031-60997-8_4.

[17] T. L. Gimenes, F. Pisani, and E. Borin, "Evaluating the performance and cost of accelerating seismic processing with CUDA, OpenCL, OpenACC, and OpenMP," 2018 IEEE International Parallel and Distributed Processing Symposium (IPDPS), Vancouver, BC, Canada, 2018, pp. 399–408. doi: 10.1109/IPDPS.2018.00050.

[18] J. Nickolls, "GPU parallel computing architecture and CUDA programming model," 2007 IEEE Hot Chips 19 Symposium (HCS), Stanford, CA, USA, 2007, pp. 1-12, doi: 10.1109/HOTCHIPS.2007.7482491.

[19] T. Nishitsuji, "Basics of OpenCL," in Hardware Acceleration of Computational Holography, T. Shimobaba and T. Ito, Eds., Singapore: Springer Nature Singapore, 2023, pp. 83–95. doi: 10.1007/978-981-99-1938-3_6.

[20] C. Zou, C. Xia and G. Zhao, "Numerical Parallel Processing Based on GPU with CUDA Architecture," 2009 International Conference on Wireless Networks and Information Systems, Shanghai, China, 2009, pp. 93-96, doi: 10.1109/WNIS.2009.46.

[21] R. Hudi, M. Silvano and K. Suganto, "Performance Evaluation of CUDA Parallel Matrix Multiplication using Julia and C++," 2024 IEEE 17th International Symposium on Embedded Multicore/Many-core Systems-on-Chip (MCSoC), Kuala Lumpur, Malaysia, 2024, pp. 349-353, doi: 10.1109/MCSoC64144.2024.00064.

[22] U. R. Khamdamov, M. A. Umarov, S. P. Khalilov, A. A. Kayumov, and F. Sh. Abidova, "Traffic sign recognition by image preprocessing and deep learning," in Lecture notes in computer science, 2024, pp. 81–92. doi: 10.1007/978-3-031-53830-8_9.

[23] M. M. Mahmudovich, A. M. Ilkhamovna and T. B. Shukhrat ogli, "Image Approach to Uzbek Speech Recognition," 2022 IEEE 22nd International Conference on Communication Technology (ICCT), Nanjing, China, 2022, pp. 1201-1206, doi: 10.1109/ICCT56141.2022.10072522.

[24] U. Khamdamov and H. Zaynidinov, "Parallel Algorithms for Bitmap Image Processing Based on Daubechies Wavelets," 2018 10th International Conference on Communication Software and Networks (ICCSN), Chengdu, China, 2018, pp. 537-541, doi: 10.1109/ICCSN.2018.8488270.

[25] Y. Cheng and B. Li, "Image Segmentation Technology and Its Application in Digital Image Processing," 2021 IEEE Asia-Pacific Conference on Image Processing, Electronics and Computers (IPEC), Dalian, China, 2021, pp. 1174-1177, doi: 10.1109/IPEC51340.2021.9421206.

[26] A. P. Singh, S. Sara, N. N. Sakhare, R. Kumar, P. K. Kumar and R. Saini, "Exploring the Application of Image Restoration Algorithms in Digital Image Processing," 2023 International Conference on Emerging Research in Computational Science (ICERCS), Coimbatore, India, 2023, pp. 1-7, doi: 10.1109/ICERCS57948.2023.10434072.

[27] A. E. Saleh, F. J. Zbeda, S. A. Salem and S. O. Albasheer, "Image Restoration Using Hybrid Technique Based on Noise Detection and Uncetainty," 2021 IEEE 1st International Maghreb Meeting of the Conference on Sciences and Techniques of Automatic Control and Computer Engineering MI-STA, Tripoli, Libya, 2021, pp. 700-704, doi: 10.1109/MI-STA52233.2021.9464503.

[28] R. R. Chand, M. Farik and N. A. Sharma, "Digital Image Processing Using Noise Removal Technique: A Non-Linear Approach," 2022 IEEE Asia-Pacific Conference on Computer Science and Data Engineering (CSDE), Gold Coast, Australia, 2022, pp. 1-5, doi: 10.1109/CSDE56538.2022.10089258.

[29] C. Ma, S. Zeng and D. Li, "A New Algorithm for Backlight Image Enhancement," 2020 International Conference on Intelligent Transportation, Big Data & Smart City (ICITBS), Vientiane, Laos, 2020, pp. 840-844, doi: 10.1109/ICITBS49701.2020.00185..

[30] Y. Zhao and Y. Liu, "Research on SAR Image Processing Performance Based on OpenMP and CUDA Parallel Model," 2022 International Conference on Computer Engineering and Artificial Intelligence (ICCEAI), Shijiazhuang, China, 2022, pp. 344-348, doi: 10.1109/ICCEAI55464.2022.00078.